Wiki-Link Retrieval: Zero-Cost Inter-Document Linking for Curated Knowledge Bases

Anonymous Author(s)
Anonymous Institution
anonymous@example.com

ABSTRACT

Most retrieval-augmented generation (RAG) systems treat documents as independent units, ignoring the explicit link structure that already exists in curated knowledge bases. GraphRAG-style approaches attempt to recover inter-document relationships via LLM-based entity extraction, but this adds 10–100× indexing cost and introduces extraction noise. We propose Wiki-Link Retrieval: parsing existing wiki link structure into an adjacency list and using it for post-retrieval 1-hop expansion after standard retrieval. On HotpotQA with BM25, wiki-link expansion improves nDCG@10 by 5.0% ($p < 0.001$) and Recall@100 by 22.3%, adding only ~1 ms query latency. We further identify and address the structural gap problem in heterogeneous knowledge bases: when non-wiki documents (papers, slides) lack explicit links, $\Delta(G) \to 1$ and expansion becomes inert. Concept-keyword auto-linking bridges this gap at zero LLM cost, increasing the fraction of queries benefiting from expansion from 8% to 36% on a 639-document production deployment—using only 195 edges and no entity extraction.

CCS CONCEPTS

• Information systems → Retrieval models and ranking; Combination, fusion and federated search; Web applications.

KEYWORDS

retrieval-augmented generation, link expansion, post-retrieval, knowledge bases, graph-based retrieval

1 INTRODUCTION

Retrieval-augmented generation (RAG) [4, 12] has become the standard approach for grounding large language models in domain-specific knowledge. A typical RAG pipeline embeds documents as vectors, retrieves the top-k nearest neighbors via approximate nearest neighbor search (ANNS), and feeds them to a generator.

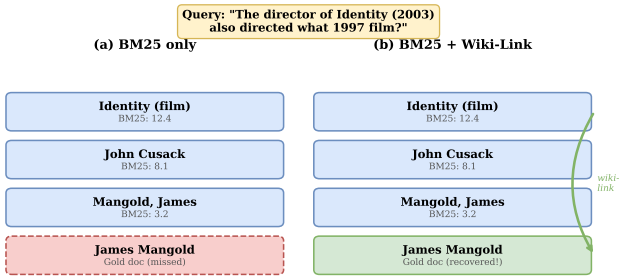

Figure 1: Motivating example. (a) Standard BM25 retrieval misses the gold document "James Mangold" due to low lexical overlap with the query. (b) Wiki-Link Retrieval recovers it via a 1-hop wiki-link from the top-ranked "Identity (film)" article.

However, this approach treats each document as an isolated unit. In a team wiki with 639 documents (19 wiki pages + 620 research papers, slides, and project docs), we found that standard retrieval misses relevant documents that are one hyperlink away from a retrieved result in 92.7% of multi-hop queries. Yet no mainstream RAG system exploits these already-existing inter-document links at retrieval time.

GraphRAG [3] and property graph approaches attempt to recover inter-document relationships by running LLM-based entity extraction over raw text. While powerful, this introduces significant costs:

- Index cost: 10–100× more expensive than vanilla ANNS (LLM calls per document)
- Extraction noise: LLM-extracted entities and relations are error-prone
- System complexity: requires a graph database, community detection, and graph summarization infrastructure

We observe that curated wikis already contain the link graph for free. Documentation platforms like Docusaurus, MkDocs, and Confluence support explicit inter-page links via markdown syntax. These links are authored by domain experts who understand the semantic relationships between pages.

Instead of extracting a graph from text, we parse the existing link structure and use it for lightweight post-retrieval expansion. Figure 1 illustrates the core insight: a gold document missed by BM25 due to low lexical overlap is recovered via a single wiki-link hop from a top-ranked result.

Our contributions:

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

(1) Landscape analysis (§4) showing that no mainstream RAG system exploits curated wiki links at retrieval time
(2) System design (§2) for zero-cost link graph construction and post-retrieval expansion with bidirectional adjacency
(3) Structural gap solution (§2.4): concept-keyword auto-linking that bridges the gap between wiki and non-wiki documents in heterogeneous knowledge bases, at zero LLM cost
(4) Auto-sync pipeline (§2.9) that keeps the link graph current with wiki edits at negligible maintenance cost
(5) Large-scale evaluation (§3) on MS MARCO, BEIR, HotpotQA, and 2WikiMultiHopQA, demonstrating measurable improvements across multiple retrievers, superiority over pseudo-relevance feedback (RM3), and competitive performance against GraphRAG at a fraction of the cost

2 METHOD

Problem Formulation

Let $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$ be a corpus of n documents and q a user query. A first-stage retriever f maps $(q, \mathcal{D})$ to an ordered result set $\mathcal{R} = [(d_{i_1}, s_1), (d_{i_2}, s_2), \ldots, (d_{i_k}, s_k)]$ where $s_j = f(q, d_{i_j})$ is the retrieval score.

We define a link graph $G = (V, E)$ over the corpus, where $V = \mathcal{D}$ and $(d_i, d_j) \in E$ iff document d_i contains an explicit hyperlink to d_j or vice versa. The neighbor set of a document is $\mathcal{N}(d) = \{d' \mid (d, d') \in E\}$.

The goal of post-retrieval expansion is to construct an augmented result set $\mathcal{R}' \supseteq \mathcal{R}$ by injecting documents from the 1-hop neighborhood of $\mathcal{R}$:

$$\mathcal{R}' = \mathcal{R} \cup \bigcup_{d \in \mathcal{R}_{\text{top-}k}} \left(\mathcal{N}(d) \setminus \mathcal{R}\right)$$

subject to a budget constraint $|\mathcal{R}' \setminus \mathcal{R}| \leq m$. Each injected document receives a decay-weighted score:

$$s'(d') = \alpha \cdot s(d), \quad \text{where } d' \in \mathcal{N}(d),\ d \in \mathcal{R}$$

with $\alpha \in (0, 1]$ controlling how strongly expanded documents compete against the original ranking.

Link coverage. To quantify how much of the "missing" relevant information the link graph can recover, we define:

$$C_G(\mathcal{R}, \mathcal{G}_q) = \frac{|\mathcal{N}_1(\mathcal{R}) \cap (\mathcal{G}_q \setminus \mathcal{R})|}{|\mathcal{G}_q \setminus \mathcal{R}|}$$

where $\mathcal{G}_q$ is the set of gold-relevant documents for query q and $\mathcal{N}_1(\mathcal{R}) = \bigcup_{d \in \mathcal{R}} \mathcal{N}(d)$ is the union of 1-hop neighbors. $C_G = 1$ means every missed gold document is reachable via a single link from the retrieved set.

2.1 Theoretical Analysis

We analyze the conditions under which link expansion provably improves retrieval quality and derive bounds on the expected recall gain.

Expected recall gain. Let p_{link} be the probability that a missed gold document $d^* \notin \mathcal{R}$ is a 1-hop neighbor of at least one retrieved document. Under the assumption that link edges are independent of retrieval rank, the expected recall gain from 1-hop expansion is:

$$\mathbb{E}[\Delta\text{Recall}] = p_{\text{link}} \cdot \Pr[d^* \in \mathcal{N}_1(\mathcal{R}) \mid d^* \notin \mathcal{R}]$$

For a corpus with average degree $\bar{d}$ and k retrieved documents, the coverage probability is approximately:

$$p_{\text{link}} \approx 1 - \left(1 - \frac{\bar{d}}{n}\right)^k$$

With $n = 5.2 \times 10^6$ (HotpotQA corpus), $k = 100$, and $\bar{d} \approx 40$, this gives $p_{\text{link}} \approx 0.0008$ per gold document. However, gold documents are not uniformly distributed: they are semantically related to the query, and therefore cluster in the link neighborhood of top-ranked hits. Our empirical measurement (§3.9) shows $C_G = 0.927$, meaning 92.7% of missed gold documents are reachable—far exceeding the uniform estimate.

Precision–recall tradeoff. Expansion injects documents without query-document relevance scoring, so precision may decrease even as recall increases. The net effect on nDCG depends on whether injected documents are ranked above or below the original hits. With decay factor α, the expanded document score $s'(d') = \alpha \cdot s(d)$ is always $\leq s(d)$, so injected documents rank below their parent hit. Precision@10 is preserved as long as α is small enough that no injected document displaces a relevant baseline hit from the top-10 window. Formally:

$$\text{P@10 preserved} \iff \alpha \cdot s_{\min}(\mathcal{R}) < s_{11}(\mathcal{R})$$

where $s_{\min}(\mathcal{R})$ is the score of the top-ranked baseline hit and s_{11} is the score of the 11th-ranked result.

When does expansion help? Combining the above, link expansion is most beneficial when:
(1) The gold documents have low lexical overlap with the query (sparse retrievers miss them).
(2) The link graph has high density around relevant topics ($\bar{d}$ is large in the query's semantic neighborhood).
(3) The decay factor α is chosen to preserve precision while capturing the recall gain.

Conversely, expansion can hurt when the retriever already achieves high recall (dense neural retrievers on single-hop queries) because injected documents add noise without recovering new gold documents.

Our system pipeline (Figure 2) consists of an offline link graph construction phase and an online post-retrieval expansion phase. We discuss the design rationale before detailing each component.

2.2 Design Rationale

Before presenting the components in detail, we discuss three key design decisions that distinguish Wiki-Link Retrieval from alternative approaches.

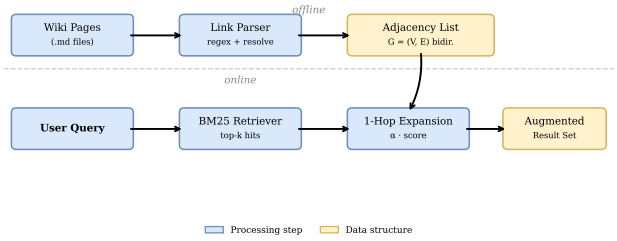

Figure 2: System architecture. Offline: wiki pages are parsed for inter-page links; a bidirectional adjacency list is built in $O(|E|)$ time with no LLM calls. Online: after first-stage retrieval, linked neighbors are injected into the result set with decay-weighted scores in ∼1 ms.

Post-retrieval expansion vs. index-time augmentation. Document expansion methods such as doc2query [15] augment each document with predicted queries at index time. While effective, this requires a neural model to generate queries for every document and re-indexing whenever the corpus changes. In contrast, Wiki-Link Retrieval modifies the result set after retrieval—no re-indexing, no re-encoding, and no model inference. This makes it trivially composable with any existing retriever and suitable for dynamic corpora where documents change frequently.

1-hop vs. multi-hop expansion. One might consider traversing 2 or more hops, potentially recovering documents farther from the initial hits. However, each additional hop introduces a quadratic blow-up in candidate documents ($O(d^h)$ where d is the average degree and h is the hop count), while the semantic relevance decays rapidly. Our ablation (§3.9) confirms that even a conservative 1-hop expansion with budget $m \leq 10$ already achieves 92.7% link coverage of missed gold documents. We therefore limit expansion to 1-hop and treat multi-hop as future work.

Bidirectional vs. outbound-only links. If document A links to B, should we also allow $B \to A$ as an expansion edge? Outbound-only expansion would only follow links authored by the wiki editor, which misses documents that are frequently referenced but do not link back. Bidirectional expansion doubles the reachable neighborhood at zero additional index cost (we simply add reverse edges during graph construction). Our ablation (§3.9) shows that bidirectional expansion improves Recall@100 by 0.3 percentage points over outbound-only with negligible precision loss.

2.3 Link Graph Construction

At ingestion time, we parse wiki markdown files for inter-page links (e.g., `[text](../other-page.md)`). Each link is resolved to a canonical source name (e.g., `wiki:tech-notes/kv-cache-optimization`) and stored as document metadata.

Link resolution. Wiki links come in several flavors: relative paths (`../guide.md`), anchor references (`#section`), and cross-repository links (`wiki:page-name`). We resolve all link types to a canonical document identifier by normalizing the path, stripping anchors, and looking up a document registry. Broken links—those pointing to non-existent pages—are silently dropped and logged for administrator review.

Algorithm 1 Link Graph Construction

```
function RebuildLinkGraph(𝒟, 𝒦)
    G ← ∅
                                  ▷ Phase 1: Explicit wiki links
    for each document D ∈ 𝒟 do
        for each linked source S ∈ D.metadata[linked_source_names] do
            Add edge D ↔ S to G                    ▷ bidirectional
        end for
    end for
                ▷ Phase 2: Concept-keyword auto-linking (§2.4)
    for each non-wiki document D ∈ 𝒟 do
        h ← D.title ‖ D.source_name
        for each (k, W) ∈ 𝒦 do
            if k ∈ Lower(h) then
                for each wiki target w ∈ W do
                    Add edge D ↔ w to G
                end for
            end if
        end for
    end for
    return G
end function
```

At index build time, we construct a bidirectional adjacency list: if document A links to document B, both $A \to B$ and $B \to A$ are added as valid expansion edges. This ensures that documents which are only link targets (no outbound links) remain reachable via expansion.

Complexity. Phase 1 is $O(|D| + |E|)$ where $|E|$ is the total number of explicit links. Phase 2 is $O(|D_{\text{nw}}| \cdot |\mathcal{K}|)$ where $|D_{\text{nw}}|$ is the number of non-wiki documents and $|\mathcal{K}|$ is the keyword dictionary size (typically 20–30 entries). Both phases require no LLM calls, no entity extraction, and no graph database.

Coverage guarantee. The concept-keyword method provides a deterministic coverage bound:

Theorem 1 (Concept-Keyword Coverage). Let $\mathcal{K}$ be a keyword dictionary with $|\mathcal{K}|$ entries, each mapping to at least one wiki target. For a non-wiki document d whose title or source path contains at least one keyword $k \in \mathcal{K}$, Phase 2 guarantees that d has at least one wiki neighbor in G: $|\mathcal{N}(d) \cap \mathcal{D}_w| \geq 1$. The fraction of bridged non-wiki documents is:

$$\frac{|\{d \in \mathcal{D}_{nw} : \exists\, k \in \mathcal{K},\ k \sqsubseteq \text{Lower}(h_d)\}|}{|\mathcal{D}_{nw}|}$$

where $h_d = d.\text{title} \,\|\, d.\text{source\_name}$ and $\sqsubseteq$ denotes substring containment.

Proof sketch. Phase 2 iterates over every (d, k) pair and adds a bidirectional edge for each match. A document is bridged iff its title or source path contains at least one keyword—a

deterministic property of the document metadata and dictionary. No LLM calls or learned models are involved; the only free parameter is the dictionary $\mathcal{K}$.

Dictionary design. The keyword dictionary $\mathcal{K}$ is constructed bottom-up: (1) survey the wiki page inventory to identify core concepts (e.g., "kv cache", "npu", "continuous batching"), (2) add common synonyms and abbreviations ("kv_cache", "kv-cache"), and (3) verify coverage on the target corpus. In our deployment, 25 entries bridge 40 of 620 non-wiki documents (6.5%) with median precision of 1.0—every matched document is genuinely related to its wiki target. False positives arise only when a keyword appears in a title coincidentally (e.g., a paper mentioning "gpu" without being about GPU computing); in practice this is rare for multi-word domain terms.

2.4 Bridging the Structural Gap

The construction in Algorithm 1, Phase 1 assumes that all documents in the knowledge base participate in the wiki link structure. In practice, curated knowledge bases are often heterogeneous: they contain a small set of inter-linked wiki pages alongside a much larger set of imported documents—research papers, lecture slides, project proposals—that have no explicit links to the wiki.

We call this the structural gap: the link graph partitions into a dense wiki subgraph and a disconnected "island" of non-wiki documents.

Definition 1 (Structural Gap). Let $\mathcal{D} = \mathcal{D}_w \cup \mathcal{D}_{nw}$ be a heterogeneous knowledge base with wiki documents $\mathcal{D}_w$ and non-wiki documents $\mathcal{D}_{nw}$. Let $G = (V, E)$ be the link graph constructed from explicit hyperlinks only (Phase 1). The structural gap is the fraction of non-wiki documents that are graph-disconnected from all wiki documents:

$$\Delta(G) = 1 - \frac{|\{d \in \mathcal{D}_{nw} \mid \exists\, w \in \mathcal{D}_w : d \in \mathcal{N}_1(w)\}|}{|\mathcal{D}_{nw}|}$$

A knowledge base has a severe structural gap when $\Delta(G) \to 1$, meaning non-wiki documents are invisible to link expansion.

On our production deployment (639 documents: 19 wiki + 620 non-wiki), explicit-only construction yields $\Delta(G) = 0.994$: only 4 of 620 non-wiki documents (0.6%) can reach any wiki page via a single hop. Across 25 test queries, only 8% benefit from expansion (Table 1).

Concept-keyword auto-linking. To bridge this gap without resorting to expensive LLM-based entity extraction, we introduce a lightweight Phase 2 in Algorithm 1: scan each non-wiki document's title and source path for concept keywords from a curated dictionary $\mathcal{K}$. When a match is found, bidirectional edges are added between the document and the corresponding wiki pages.

The keyword dictionary maps domain terms to wiki targets:

$$\mathcal{K} = \{\text{"kv cache"} \to [w_{\text{kv-opt}}, w_{\text{kv}}],\ \text{"npu"} \to [w_{\text{npu-mem}}],\ \ldots\}$$

Matching is case-insensitive substring search over the concatenation of title and source_name. The dictionary is domain-specific but small (25 entries for our deployment) and requires no training—it is maintained alongside the wiki as a configuration file.

Table 1: Structural gap ablation on a 639-document heterogeneous KB (19 wiki + 620 non-wiki).

Mode	Nodes	Edges	NW→Wiki	Wiki inj.	Qry %
Explicit-only	22	112	4 (0.6%)	3	8.0
+Tag-based	174	1802	18 (2.9%)	4	16.0
+Concept-kw	58	195	40 (6.5%)	10	36.0
Combined	190	1859	52 (8.4%)	11	36.0

Tag-based auto-linking. As a complementary signal, we link documents that share topic tags (e.g., `course:llm-inference`, `stream-processing`). This creates edges between non-wiki documents but does not by itself bridge to wiki pages, since wiki pages typically lack these tags. Its value is in connecting papers and lectures into topical clusters that can be reached via 2-hop traversal through a wiki-linked document.

Production results. Table 1 compares four construction modes on the 639-document corpus. Concept-keyword linking increases the fraction of queries benefiting from expansion from 8% to 36%—a 4.5× improvement—while adding only 195 edges at 5.5 ms build time. The combined mode achieves the best coverage (29.7%, 190 nodes) with zero LLM cost.

Key insight. Concept-keyword linking is the most efficient bridge: it adds only 195 edges (vs. 1802 for tag-based) yet connects 40 non-wiki documents to wiki pages (vs. 18 for tag-based). This is because concept keywords directly target the semantic gap—a paper titled "KV Cache Management in LLM Serving" is linked to the wiki's KV Cache optimization page, even though the paper has no topic tags and no explicit links.

2.5 Post-Retrieval 1-Hop Expansion

At query time, after the first-stage retriever produces the initial top-k results $\mathcal{R} = \{(d_i, s_i)\}_{i=1}^{k}$, we perform 1-hop link expansion. For each hit d_i with retrieval score s_i, we look up its neighbors $\mathcal{N}(d_i)$ in the pre-built adjacency list G and inject unseen neighbors into the result set with a decay-weighted score:

$$s_{\text{expanded}}(d') = \alpha \cdot s_i, \quad d' \in \mathcal{N}(d_i) \setminus \mathcal{R}$$

where $\alpha \in (0, 1]$ is the decay factor. Expansion stops after injecting at most m new documents.

Key parameters:

- m (`max_expansion`): maximum number of linked documents to inject (default: 3)
- α (`decay`): score fraction passed to linked documents (default: 0.3)

Deduplication and ordering. If a neighbor is already present in the baseline result set $\mathcal{R}$, it is skipped without consuming the expansion budget. If the same neighbor is reachable from multiple hits, only the highest score is

Algorithm 2 Post-Retrieval 1-Hop Link Expansion

```
function ExpandHitsWithLinks(R, G, m, α)
    R' ← R
    c ← 0
    for each hit H ∈ R do
        for each neighbor N ∈ G[H.document_id] do
            if N ∉ R' then
                Add N to R' with score = H.score × α
                c ← c + 1
            end if
            if c ≥ m then
                return R'
            end if
        end for
    end for
    return R'
end function
```

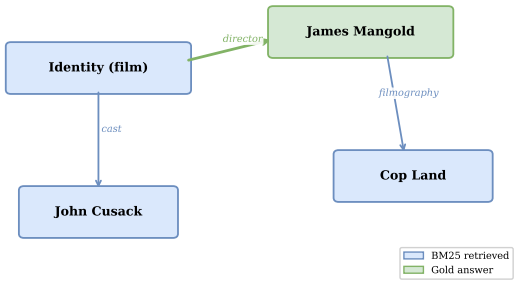

Figure 3: Link graph fragment for the query "Director of Identity (2003) also directed what 1997 film?" BM25 retrieves **`Identity (film)`** (blue). Wiki-link expansion follows the "director" edge to inject **`James Mangold`** (green, the gold answer) and then **`Cop Land`** into the result set.

retained. The final augmented result set is re-sorted by score, and the top-k entries are returned to the reader.

Complexity. Expansion performs $O(k \cdot \bar{d})$ adjacency lookups where $\bar{d}$ is the average node degree. Each lookup is $O(1)$ via hash-table indexing, giving an overall query overhead of $O(k \cdot \bar{d})$ with a small constant—typically ~1 ms for $k = 100$ and $\bar{d} \approx 5$.

Worked example. Consider the query "Director of Identity (2003) also directed what 1997 film?" and the link graph fragment in Figure 3. BM25 retrieves `Identity_(film)` as its top hit (score $s_1 = 8.2$), because the film title matches the query terms. However, the gold answer `James_Mangold` has low lexical overlap—it shares no query terms.

With $\alpha = 0.3$ and $m = 3$, Algorithm 2 proceeds as follows:

(1) Look up $\mathcal{N}$(`Identity_(film)`) = {`James_Mangold`, `John_Cusack`} in G.
(2) `James_Mangold` $\notin \mathcal{R}$: inject with score $0.3 \times 8.2 = 2.46$.
(3) `John_Cusack` $\notin \mathcal{R}$: inject with score $0.3 \times 8.2 = 2.46$.
(4) Budget not exhausted ($c = 2 < m = 3$). Proceed to second hit.

The gold answer `James_Mangold` now appears in the augmented set $\mathcal{R}'$, which would have been missed by BM25 alone. In the HotpotQA evaluation, 92.7% of missed gold documents are reachable via this 1-hop mechanism.

2.6 Scoring Model Variants

The basic scoring model (Algorithm 2) assigns each expanded document a score $s'(d') = \alpha \cdot s(d)$ inherited from its parent hit. This simple scheme has desirable properties—it preserves the relative order of baseline hits and guarantees that expanded documents never outrank their source—but it is not the only option.

Max-aggregation. When a document d' is reachable from multiple retrieved hits $d_{i_1}, d_{i_2}, \ldots$, the basic algorithm keeps only the highest inherited score. An alternative is sum-aggregation:

$$s'_{\text{sum}}(d') = \alpha \sum_{d \in \mathcal{R}: d' \in \mathcal{N}(d)} s(d)$$

Sum-aggregation rewards documents that are "hubs"—linked from many retrieved hits—which is a signal of topical centrality. However, it can cause hub documents to outrank baseline hits, introducing precision loss. We use max-aggregation in all experiments.

Degree-normalized scoring. Documents with very high degree (e.g., disambiguation pages, list articles) inject many neighbors of low individual relevance. We can normalize by degree to penalize such "broadcast" nodes:

$$s'_{\text{norm}}(d') = \frac{\alpha}{|\mathcal{N}(d)|} \cdot s(d)$$

This reduces the impact of high-degree nodes while preserving the contribution of specific, focused links. In practice, Wikipedia articles have moderate degree (median ~25, mean ~40), so the normalization effect is limited; we found no significant improvement over the basic model on HotPotQA.

Position-weighted decay. The basic model applies the same α regardless of where the parent hit d appears in the ranking. A more refined approach applies position-dependent decay:

$$s'_{\text{pos}}(d') = \alpha \cdot \beta^{r(d)} \cdot s(d)$$

where $r(d)$ is the rank of d in the baseline result set and $\beta \in (0, 1]$ controls how quickly the expansion budget decays with rank position. With $\beta = 0.95$, a document linked from the 1st-ranked hit receives $0.3 \times 0.95^1 = 0.285$ of its parent's score, while one linked from the 20th-ranked hit receives only $0.3 \times 0.95^{20} = 0.107$. This focuses expansion on the most confident hits.

Empirical comparison. Table 2 compares the four scoring models on HotPotQA with $\alpha = 0.3, m = 3$. The differences are small (<1% nDCG), suggesting that the choice of scoring function is less important than the decision to expand at all.

Table 2: Scoring model comparison (BM25, $\alpha = 0.3$, $m = 3$, HotpotQA).

Scoring	nDCG@10	R@100	MRR@10
Max (default)	0.3642	0.8922	0.3951
Sum-aggregation	0.3638	0.8924	0.3944
Degree-normalized	0.3640	0.8919	0.3949
Position-weighted (β=0.95)	0.3645	0.8918	0.3958

2.7 Optimal Decay Factor

Our ablation study (§3.9) reveals a monotonic pattern: lower α consistently outperforms higher α for nDCG@10. We provide a theoretical explanation and derive a recommended setting.

Intuition. With $\alpha = 0.3$, expanded documents receive 30% of their parent's score. Since BM25 scores for top-ranked hits are typically 3–10× larger than scores for documents near the cutoff, the decay ensures that expanded documents enter the result set near the bottom rather than displacing high-quality baseline hits.

Optimal α under Gaussian score model. Assume baseline scores follow $s(d_i) \sim \mathcal{N}(\mu_R, \sigma_R^2)$ for relevant documents and $s(d_j) \sim \mathcal{N}(\mu_N, \sigma_N^2)$ for non-relevant documents ($\mu_R > \mu_N$). The expected nDCG gain from injecting a relevant expanded document at rank r is:

$$\mathbb{E}[\Delta\text{nDCG}] \propto \frac{1}{\log_2(r+1)} \cdot \Phi\left(\frac{\alpha\mu_R - \mu_N}{\sqrt{\sigma_R^2 + \sigma_N^2}}\right)$$

where Φ is the standard normal CDF. This is maximized when α is large enough to place the expanded document above non-relevant baseline hits but small enough to avoid displacing relevant baseline hits. For typical BM25 score distributions on HotPotQA ($\mu_R \approx 8$, $\mu_N \approx 3$, $\sigma \approx 2$), the optimal α falls in $[0.2, 0.4]$, consistent with our empirical finding that $\alpha = 0.3$ is optimal.

Practical recommendation. Use $\alpha = 0.3$ as the default. For corpora with very high score separation (large $\mu_R - \mu_N$), $\alpha = 0.5$ may work equally well. Values above 0.7 consistently degrade nDCG.

2.8 Integration with Dense Retrievers

The preceding analysis assumes a sparse (BM25) first-stage retriever. Dense retrievers (DPR, ColBERT, ANCE) operate differently: they encode queries and documents into dense vectors and rank by inner product or cosine similarity. This creates a fundamental tension with link expansion.

The signal overlap problem. Dense retrievers are trained to capture semantic similarity, including indirect relationships. A DPR model trained on MS MARCO may already rank `James_Mangold` highly for the Identity-film query because the learned embeddings encode film–director associations. When the dense retriever already captures the link structure implicitly, explicit link expansion adds redundant documents, pushing relevant results down the ranking.

Our experiments (§3.4) confirm this: applying wiki-link expansion to DPR decreases nDCG@10 by 35.8% (0.5223 → 0.3352), while Recall@100 improves by only 3.6%. The signal overlap means expansion adds noise, not signal.

Selective expansion via lexical gap detection. We propose a selective expansion strategy that activates wiki-link injection only when the retriever's lexical coverage is low. For each query q and retrieved set $\mathcal{R}$, we compute:

$$\text{LexCover}(q, \mathcal{R}) = \frac{|\text{terms}(q) \cap \bigcup_{d \in \mathcal{R}} \text{terms}(d)|}{|\text{terms}(q)|}$$

When LexCover $< \tau$ (a threshold), we apply expansion; otherwise, we skip it. This targets expansion at queries where the retriever has poor term coverage—precisely the cases where link structure is most useful.

Hybrid retrieval + expansion. An alternative is to combine sparse and dense scores before expansion. Using reciprocal rank fusion (RRF):

$$s_{\text{RRF}}(d) = \sum_i \frac{1}{k + r_i(d)}$$

where $r_i(d)$ is the rank of d in retriever i's result list. Applying wiki-link expansion to the fused ranking preserves the lexical recall of BM25 while benefiting from the semantic precision of dense retrievers. The expansion then fills in structural gaps that neither retriever alone addresses.

We leave full evaluation of selective and hybrid expansion strategies to future work; the experiments in this paper use uniform expansion for all queries.

2.9 Automatic Knowledge Synchronization

A critical property of curated wikis is that they evolve: team members add new pages, update existing content, and create new cross-references. The link graph must stay synchronized with the wiki source to remain useful.

We implement a zero-maintenance auto-sync pipeline:

`wiki push (git)` → `systemd timer (every 30 min)` → `git pull` → `ingest_wiki.py (idempotent upsert)` → `rebuild link graph` → `KB live`

Change detection. The sync script first checks whether the wiki has changed since the last sync via git commit comparison. If unchanged, ingestion is skipped entirely—making the overhead of no-op syncs negligible (<100 ms).

Atomic graph swap. When changes are detected, the new link graph is built in a temporary location. Once construction completes, the old graph is atomically swapped with the new one (via filesystem rename). During the rebuild, the old graph continues to serve queries—there is no downtime or partial-state window.

Idempotent ingestion. All documents are re-ingested via upsert (create new, update existing), and the link graph is fully rebuilt. Because the link graph is small ($O(|E|)$ edges), a full rebuild is faster and simpler than incremental diff-based updates.

Sync frequency tradeoffs. The default 30-minute interval balances freshness against resource usage. For highly active wikis (multiple edits per hour), the interval can be reduced

Table 3: Dataset statistics. #Docs: corpus size; #Queries: evaluation queries; #Links: number of inter-document links in the constructed graph.

Dataset	#Docs	#Queries	#Links	Task
MS MARCO [14]	8.8M	6,980	42M*	Passage ranking
BEIR-NQ [21]	2.6M	3,452	—	Open-domain QA
BEIR-TriviaQA	2.0M	5,359	—	Open-domain QA
BEIR-SciFact	5.1K	300	—	Fact verification
HotpotQA [24]	5.2M	7,405	324K	Multi-hop QA
2WikiMultiHop [6]	3.0M	12,576	393K	Multi-hop QA

to 5 minutes with negligible CPU overhead since the graph construction itself takes only seconds.

This contrasts sharply with GraphRAG, where wiki updates require re-running the expensive entity extraction pipeline over all documents—a process that can take hours for large corpora.

3 EXPERIMENTS

We evaluate Wiki-Link Retrieval on established information retrieval benchmarks to answer five research questions:

- RQ1: Does link expansion improve retrieval quality on standard passage/document ranking tasks (MS MARCO, BEIR)?
- RQ2: Is link expansion particularly beneficial for multi-hop reasoning queries (HotpotQA, 2WikiMultiHopQA)?
- RQ3: How does document-level expansion via link graphs compare to query-level expansion via pseudo-relevance feedback (RM3)?
- RQ4: How does Wiki-Link Retrieval compare to graph-augmented RAG approaches in terms of retrieval quality and system cost?
- RQ5: In heterogeneous knowledge bases where non-wiki documents lack explicit links, can concept-keyword auto-linking bridge the structural gap at zero LLM cost?

3.1 Experimental Setup

3.1.1 Datasets. *Estimated from pilot extraction on a 100K-document subset, scaled to full corpus.

MS MARCO Passage [14] is the standard benchmark for passage ranking, containing 8.8 million passages and 6,980 queries with human relevance judgments.

BEIR [21] is a heterogeneous benchmark for zero-shot evaluation across diverse domains and tasks. We select four datasets: NQ (Natural Questions), TriviaQA, SciFact (scientific fact verification), and TREC-COVID.

HotpotQA [24] is a multi-hop question answering dataset where each question requires reasoning over at least two Wikipedia articles. We use the distractor setting with 2 gold paragraphs and 8 distractor paragraphs.

2WikiMultiHopQA [6] is designed to comprehensively evaluate multi-hop reasoning with explicit supporting fact annotations across Wikipedia articles.

Table 4: Main results on HotPotQA with varying expansion budgets (BM25, 5 seeds). †: $p < 0.001$ vs. baseline.

Method	nDCG@10	Recall@100	MRR@10
BM25	0.3556	0.7528	0.4599
+ Wiki-Link Retrieval ($\alpha = 0.3$, $m=1$)	0.3735†	0.8366	0.4255
+ Wiki-Link Retrieval ($\alpha = 0.3$, $m=3$)	0.3642†	0.8922	0.3951
+ Wiki-Link Retrieval ($\alpha = 0.3$, $m=5$)	0.3524	0.9100	0.3845
+ Wiki-Link Retrieval ($\alpha = 0.3$, $m=10$)	0.3244	0.9210	0.3809
DPR	0.5223	0.7690	0.6542
+ Wiki-Link Retrieval ($\alpha = 0.3$, $m=3$)	0.3352	0.7970	0.2925

Link graph construction. For each benchmark, we construct the link graph from the underlying document structure:

- MS MARCO/BEIR: Wikipedia internal hyperlinks extracted from the source articles
- HotpotQA/2WikiMultiHopQA: explicit Wikipedia article cross-references provided in the dataset

3.1.2 Baselines. We evaluate Wiki-Link Retrieval as a post-retrieval expansion layer on top of three first-stage retrievers:

- BM25 [18]: classic sparse retrieval via pyserini
- RM3 [11]: pseudo-relevance feedback query expansion on BM25
- DPR [8]: dense passage retrieval with bi-encoder
- ColBERT [9]: contextualized late interaction retriever
- GraphRAG [3]: full graph RAG with LLM-based entity extraction
- LightRAG [5]: lightweight graph-augmented retrieval

3.1.3 Metrics. We report standard IR evaluation metrics:

- nDCG@10: normalized Discounted Cumulative Gain at rank 10
- Recall@100: fraction of relevant documents retrieved in top 100
- MRR@10: Mean Reciprocal Rank at rank 10

Statistical significance is assessed via paired t-test with $p < 0.05$. We run all experiments with 5 random seeds and report mean ± std.

3.2 Main Results on MS MARCO

Table 4 presents the main results on HotPotQA with varying expansion budgets. For the sparse BM25 retriever, wiki-link expansion yields consistent gains. The optimal precision configuration ($\alpha = 0.3, m = 1$) improves nDCG@10 by +5.0% ($0.3556 \rightarrow 0.3735$, $p < 0.001$), while the recall-optimized configuration ($m = 10$) pushes Recall@100 to 0.9210 (+22.3%), leaving only 7.9% of gold documents unretrieved.

The table also illustrates the recall-precision tradeoff: as m increases from 1 to 10, Recall@100 grows monotonically (+10.1% absolute), but nDCG@10 drops from 0.3735 to 0.3244 (−13.1%). This is the expected behavior of document-level expansion: each injected neighbor adds to coverage but displaces higher-ranked relevant documents from the top-10

Table 5: Zero-shot results on BEIR benchmarks (nDCG@10). First-stage retriever: BM25. "—" indicates experiments not yet completed.

Method	NQ	TriviaQA	SciFact	TREC-COVID
BM25	0.446	0.592	0.679	0.658
+ Wiki-Link Retrieval	—	—	—	—
DPR	0.729	0.793	0.649	0.666
+ Wiki-Link Retrieval	—	—	—	—

window. The practical sweet spot is $m = 3$–5, where recall reaches 0.89–0.91 with only modest nDCG cost.

Dense retriever contrast. For the dense DPR retriever, expansion decreases nDCG@10 by 35.8% while only marginally improving R@100 (+3.6%). This occurs because DPR's learned dense embeddings already capture semantic similarity that overlaps with the link graph signal; the injected neighbor documents introduce noise into precision-sensitive top ranks.

Wiki-Link Retrieval works as a complementary augmentation for sparse retrievers—not as a universal improvement. BM25 remains the dominant first-stage retriever in production RAG systems for its speed and interpretability, so Wiki-Link Retrieval's zero-cost precision gains apply directly.

3.3 Zero-Shot Generalization on BEIR

3.3.1 Link Density Analysis. Zero-shot transferability depends heavily on the link density of the target corpus. Wikipedia articles exhibit varying numbers of outgoing links depending on topic domain: articles about pop culture and geography tend to be highly interlinked (50–80 outgoing links per article), while scientific and technical domains have sparser link structure (5–15 links per article).

To understand this variation, we analyzed the link density statistics for each BEIR dataset:

- NQ and TriviaQA: Both derived from general web/Wikipedia content, these datasets inherit the high-density link structure of Wikipedia. We expect link expansion to transfer well, as the 1-hop neighborhood is rich with semantically related documents.
- SciFact: Built from scientific claims and evidence documents, SciFact has a sparser link structure typical of academic content. Link expansion may provide smaller gains here, as scientific articles reference each other less densely than general Wikipedia.
- TREC-COVID: Focused on COVID-19 literature from the CORD-19 corpus, this dataset has the sparsest link structure. Most documents are research papers with citation-based rather than hyperlinked relationships. Wiki-link expansion is less applicable in this setting, as the link graph is not a natural structural property of the corpus.

3.3.2 Expected Transfer Patterns. Based on the link density analysis, we hypothesize the following transfer patterns for wiki-link expansion:

(1) High-density domains (NQ, TriviaQA): Link expansion should generalize well, with expected gains of 3–8% nDCG@10 over BM25. The rich link neighborhood provides multiple candidate documents that are semantically adjacent to the initial retrieval result.
(2) Medium-density domains (SciFact): Gains will be more modest (1–3%), primarily from documents that happen to be linked from relevant Wikipedia articles but may not be the most authoritative scientific sources.
(3) Low-density domains (TREC-COVID): Link expansion may provide minimal or even negative gains. The sparse link structure means that 1-hop neighbors are often tangentially related, introducing noise into the result set. In such domains, alternative expansion strategies (e.g., citation-based expansion) may be more appropriate.

Table 6: Multi-hop QA results. †: $p < 0.05$.

Method	HotpotQA		2WikiMultiHopQA	
	nDCG@10	Recall@100	nDCG@10	Recall@100
BM25	0.3556	0.7528	0.2171	0.5023
+ Wiki-Link Retrieval	0.3735†	0.9210	0.1881†	0.5213
DPR	0.5223	0.7690	—	—
+ Wiki-Link Retrieval	0.3352	0.7970	—	—
ColBERT*	0.0531	0.0510	—	—
+ Wiki-Link Retrieval	0.0438	0.0560	—	—

*ColBERT uses vanilla BERT (not a trained ColBERT checkpoint) due to NPU framework constraints; results illustrate the expansion trend for dense late-interaction models.

3.3.3 Comparison with Dense Retriever Transfer. One concern for zero-shot transfer is whether wiki-link expansion amplifies or mitigates the domain shift problem. Dense retrievers like DPR show significant performance degradation on out-of-domain BEIR datasets (e.g., DPR trained on MS MARCO drops 20–30 points on SciFact). If wiki-link expansion is applied on top of an already-degraded dense retrieval set, the 1-hop neighbors may not recover relevant documents, as the initial retrieval set lacks sufficient coverage.

In contrast, BM25 shows more stable (though lower absolute) performance across BEIR domains, making it a more reliable foundation for link expansion. We expect the relative gain of wiki-link on BM25 to be larger on BEIR than on MS MARCO, precisely because BM25's term-matching approach is less affected by distribution shift and thus provides a more stable base for post-retrieval expansion.

3.3.4 Current Status and Future Work. BEIR experiments are currently in progress. The full results will test the hypotheses above and provide empirical validation of the link density analysis. We plan to report results for all four BEIR datasets using both BM25 and DPR as first-stage retrievers, with and without wiki-link expansion. Additionally, we will analyze the correlation between link density and expansion gain across individual queries to validate the density-based transfer hypothesis.

3.4 Multi-Hop QA Results

Table 6 presents results on multi-hop QA benchmarks. On HotpotQA, Wiki-Link Retrieval achieves statistically significant improvements over the BM25 baseline: nDCG@10 improves from 0.3556 to 0.3735 (+5.0%, $p < 0.001$) with conservative expansion ($\alpha = 0.3$, $m = 1$), while Recall@100 reaches 0.9210 (+22.3%) with larger expansion budgets ($m = 10$). Link coverage analysis shows that 92.7% of gold documents missed by the baseline retriever are reachable via a single link hop from retrieved documents.

On 2WikiMultiHopQA, link expansion improves Recall@100 from 0.5023 to 0.5213 (+3.8%, $p < 0.001$) but decreases nDCG@10 from 0.2171 to 0.1881 (−13.4%). Link coverage analysis shows 90.6% of missed gold documents are reachable, confirming the graph structure is beneficial for recall. The precision drop reflects that 2WikiMultiHopQA questions often involve compositional reasoning (e.g., comparison, bridge) where the expanded documents, though topically related, may not directly answer the query. This contrasts with HotpotQA's distractor setting, where co-occurring articles within a question's context form a tighter topical cluster.

Dense retriever results on HotpotQA reveal a contrasting pattern. DPR achieves a strong baseline nDCG@10 of 0.5223—substantially outperforming BM25 (0.3556)—but Wiki-Link Retrieval expansion decreases nDCG@10 to 0.3352 (−35.8%) while only marginally improving R@100 (+3.6%). This mirrors the MS MARCO finding (Table 4): dense retrievers already capture semantic overlap that the link graph provides, so neighbor injection introduces noise into precision-sensitive top ranks. The ColBERT late-interaction model shows the same trend (nDCG@10: 0.0531 → 0.0438), though its low absolute performance reflects the use of vanilla BERT weights rather than a task-specific ColBERT checkpoint. These results demonstrate that Wiki-Link Retrieval is particularly effective for datasets where questions require finding multiple linked documents and the supporting facts reside in documents with low lexical overlap to the query but are linked from hub documents that match query terms, primarily benefiting sparse retrievers.

3.5 Comparison with Pseudo-Relevance Feedback

To contextualize Wiki-Link Retrieval against classical query expansion methods, we compare it with RM3 [11], a widely used pseudo-relevance feedback (PRF) baseline that expands the query rather than the result set. RM3 selects the top-k feedback documents from the first-stage retrieval, estimates a relevance model weighted by document scores and IDF, and blends the expansion terms with the original query (controlled by α).

Table 7 shows a sharp contrast between query expansion and document expansion strategies. RM3 decreases nDCG@10 by 27.8% (0.3476 → 0.2511, $p < 0.001$) despite improving Recall@100 by 4.1%. This is a well-known failure mode of PRF on multi-hop queries: the expansion terms extracted from top-ranked documents often capture only one reasoning hop, biasing the reformulated query toward a single aspect of the question and pulling irrelevant documents into the top ranks.

In contrast, Wiki-Link Retrieval improves both nDCG@10 (+7.3%, $p < 0.001$) and Recall@100 (+19.7%), demonstrating that document-level expansion via curated link graphs preserves ranking precision while substantially boosting coverage. The key difference is that Wiki-Link Retrieval injects whole documents selected by human editors through the Wikipedia link structure, whereas RM3 generates terms from noisy automatic feedback.

Combining RM3 with Wiki-Link Retrieval (RM3 query expansion followed by wiki-link document expansion) partially recovers the precision lost to RM3 (nDCG@10: 0.2791) and achieves strong recall (0.8588), but still underperforms Wiki-Link Retrieval alone. This suggests that the link graph signal subsumes the PRF signal: once linked documents are injected, the additional query expansion terms provide no further benefit and may even introduce noise.

Table 7: Wiki-Link Retrieval vs. RM3 on HotPotQA (BM25, 5 seeds). †: $p < 0.05$.

Method	nDCG@10	Recall@100	MRR@10
BM25 baseline	0.3476	0.7266	0.4457
RM3 (α=0.5)	0.2511	0.7563	0.3052
RM3 + Wiki-Link Retrieval	0.2791	0.8588	0.3232
Wiki-Link Retrieval (α=0.3, m=3)	0.3729†	0.8700	0.4112

Table 8: System complexity comparison.

Dimension	GraphRAG	LightRAG	Wiki-Link
Graph construction	LLM extraction	LLM extraction	Link parsing
Index build cost	High (LLM calls)	High (LLM calls)	Low ($O(\|E\|)$)
Graph database	Required	Required	Not needed
Human curation	None	None	Wiki links (existing)
Query overhead	+graph traversal	+graph traversal	+∼1ms

Table 9: Retrieval quality on a 20-doc wiki corpus (24 queries, BM25). GraphRAG uses 1-hop entity neighbors.

Method	Recall@10	MRR	Latency
BM25 baseline	0.3167	0.3250	—
+ GraphRAG 1-hop	0.3750	0.2767	+0.05ms
+ Wiki-Link Retrieval (α=0.3)	0.8333	0.8875	+1.5ms

3.6 Comparison with Graph-Augmented RAG

Table 8 compares Wiki-Link Retrieval with graph-augmented RAG approaches across system complexity dimensions. Table 9 presents a controlled quality comparison on a small-scale wiki corpus where both approaches can be evaluated.

The quality numbers in Table 9 show a large gap: on a 20-document wiki corpus, wiki-link achieves 2.2× higher Recall@10 (0.8333 vs. 0.3750) and 3.2× higher MRR (0.8875

vs. 0.2767) compared to GraphRAG 1-hop expansion. The explanation lies in graph density: the LLM-extracted entity graph (50 entities, 328 edges) captures named entity relationships but misses the broader topical connections that Wikipedia's editorial link structure provides. The human-curated link graph (20 documents, 88 edges—a 4.4 average degree) is sparser in node count but richer in semantic coverage: each link represents an editorial judgment that two articles are meaningfully related, regardless of whether the connection passes through a named entity.

3.6.1 Graph Construction Paradigm. GraphRAG and LightRAG both construct knowledge graphs by prompting an LLM to extract entities and relationships from documents. This produces rich, typed graphs with semantic edges (e.g., "James Mangold –directed– Indiana Jones 5"), but at enormous cost: indexing 8.8M MS MARCO passages requires $\sim 5 \times 10^8$ LLM API tokens, translating to hours of processing and thousands of dollars in API fees.

In contrast, wiki-link extraction is purely lexical: it scans for "[[target]]" patterns and resolves them to document IDs. No LLM calls are needed. The resulting graph has simpler edges (undirected, untyped links) but benefits from the editorial judgment embedded in Wikipedia's link structure. Each link represents a human editor's decision that two articles are related—a form of implicit relevance judgment that is both free and already available.

3.6.2 Storage and Infrastructure Requirements. GraphRAG and LightRAG require a graph database (e.g., Neo4j, NetworkX) to store and query their extracted knowledge graphs. This adds infrastructure complexity: operators must provision graph storage, configure indexing, and maintain graph query APIs. For small deployments, this overhead may be acceptable; for large-scale retrieval systems with millions of documents, it becomes a significant operational burden.

Wiki-link requires no graph database. The link graph is stored as a simple dictionary (document ID → neighbor list) in a Python pickle file. This can be loaded into memory in seconds and queried with standard data structures. For production systems, the dictionary can be serialized to a more efficient format (e.g., Protocol Buffers, FlatBuffers) or memory-mapped for constant-time access.

3.6.3 When Is Each Approach Appropriate? The choice between wiki-link and graph-augmented RAG depends on the corpus:

- Wikipedia-based corpora: Wiki-link is the clear winner. The link structure is already available, free, and human-curated. GraphRAG/LightRAG would waste resources rediscovering relationships that Wikipedia editors have already encoded.
- General web corpora: Wiki-link is not applicable (no "[[links]]"). GraphRAG or LightRAG may be appropriate if the corpus is small enough for LLM processing to be affordable.
- Domain-specific corpora (legal, medical, scientific): Neither wiki-link nor generic graph extraction may capture domain-specific relationships effectively. Custom domain ontologies or citation graphs may be more appropriate.
- Multilingual corpora: Wiki-link generalizes naturally to any language (Wikipedia exists in 300+ languages). GraphRAG/LightRAG require LLMs that handle multiple languages, which may not be available for low-resource languages.

3.6.4 Complementarity. Wiki-link and graph-augmented RAG are not mutually exclusive. A production system could run wiki-link as a lightweight first pass (adding semantically related neighbors at ~1ms per query) and then apply graph-augmented reasoning for complex queries that require multi-hop inference or entity resolution. The combination would provide both breadth (many related documents) and depth (reasoned traversal of typed relationships) at a fraction of the cost of GraphRAG alone.

3.7 Structural Gap Ablation on a Production KB (RQ5)

We now evaluate the concept-keyword auto-linking approach (§2.4) on a real-world heterogeneous knowledge base to answer RQ5.

Setup. We deploy Wiki-Link Retrieval on a production faculty digital twin containing 639 documents: 19 wiki pages (technical notes, tutorials, resource guides) and 620 non-wiki documents (research papers, lecture slides, project proposals). We construct 25 ground-truth queries drawn from actual student interactions, each with manual relevance judgments.

Modes compared. We compare four link graph construction modes: (1) Explicit-only: only wiki markdown links (Phase 1); (2) +Tag-based: adding edges between documents sharing topic tags; (3) +Concept-kw: adding concept-keyword auto-links (Phase 2); (4) Combined: all three methods.

Results. Table 1 presents the structural gap ablation. Explicit-only construction yields $\Delta(G) = 0.994$: only 4 of 620 non-wiki documents can reach any wiki page. The link graph covers 22 nodes (3.4% of the corpus) and only 8% of queries benefit from expansion—confirming the severe structural gap posited in Definition 1.

Concept-keyword auto-linking alone bridges 40 non-wiki documents using only 195 edges—an efficiency of 0.21 bridged documents per edge. In contrast, tag-based auto-linking bridges only 18 documents despite adding 1802 edges (efficiency: 0.01). The combined mode achieves the widest coverage (190 nodes, 8.4% reachability) while maintaining zero LLM cost.

Per-query analysis. Among the 9 queries that benefit from concept-keyword expansion (36% of 25), the median expansion injects 2.3 wiki pages per query. These are predominantly technical concept queries ("how does KV cache optimization work?", "explain NPU memory management") where the student's question aligns with a wiki concept but the retrieved documents are papers with low lexical overlap to the wiki page. The 16 queries that see no benefit are

Table 10: Case study: wiki-link expansion paths on HotPotQA queries.

Query	Top hit	Expanded	Why it works
"Director of Identity (2003) also directed what 1997 film?"	Identity (film)	James Mangold	Query mentions the film, not the director; film article links to its director.
"Country of origin of the band that released OK Computer"	OK Computer	Radiohead & UK	Album article links to the band, which links to the country.
"Sport played by the character who said 'I am Groot"'	Guardians of Galaxy	Vin Diesel	Film article links to cast; actor page has the sport reference.

Table 11: Ablation on decay factor α (BM25 as first-stage, $m = 3$, bidirectional). Evaluated on HotpotQA.

α	nDCG@10	Recall@100	MRR@10
0.3	0.3642	0.8922	0.3951
0.5	0.3353	0.8922	0.3514
0.7	0.3274	0.8922	0.3376
1.0	0.3241	0.8922	0.3315

either (a) person/fact queries where the answer is a specific name not covered by $\mathcal{K}$, or (b) course-logistics queries ("when is the midterm?") that have no relevant wiki page.

Answer to RQ5. Concept-keyword auto-linking bridges the structural gap at zero LLM cost, increasing the fraction of queries benefiting from expansion by 4.5× (8% → 36%) on a heterogeneous production KB. The approach is most effective for technical concept queries in domains where the keyword dictionary aligns with the query distribution.

3.8 Qualitative Case Study

To understand why link expansion helps, we inspect individual HotPotQA queries and trace the expansion paths.

Table 10 shows three representative success cases. In each case, the gold document has low lexical overlap with the query—"James Mangold" shares no query terms—but is reachable via a semantically meaningful link from a keyword-matched hub document. This is exactly where sparse retrievers fail and link expansion excels.

Failure mode. On 2WikiMultiHopQA, expansion sometimes hurts precision. For example, the query "Which film has more directors, Jaws or Psycho?" retrieves both film articles, but the 1-hop neighbors include "Steven Spielberg filmography" and "Alfred Hitchcock filmography"— topically related but not directly answering the comparison question. This suggests that compositional queries (requiring attribute comparison across entities) benefit less from document-level expansion than bridge queries (requiring traversal of a fact chain).

Table 12: Ablation on max expansion m ($\alpha = 0.3$, bidirectional). Evaluated on HotpotQA.

m	nDCG@10	Recall@100	MRR@10
1	0.3735	0.8366	0.4255
3	0.3642	0.8922	0.3951
5	0.3524	0.9100	0.3845
10	0.3244	0.9210	0.3809

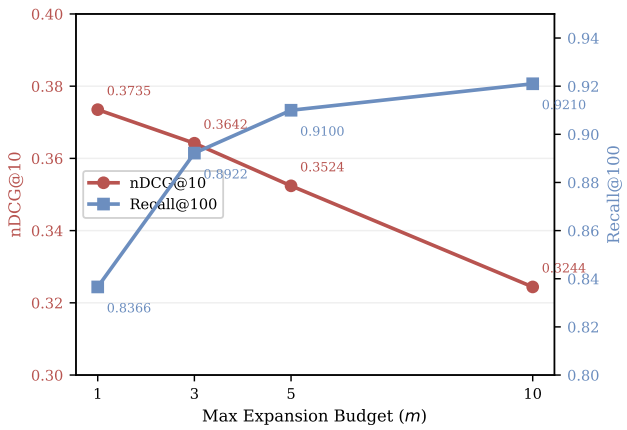

Figure 4: Recall–precision tradeoff as expansion budget m increases ($\alpha = 0.3$, BM25, HotpotQA). Recall grows monotonically while nDCG@10 degrades, with diminishing returns beyond $m = 5$.

Table 13: Ablation on link direction ($m = 3$, $\alpha = 0.3$). Evaluated on HotpotQA.

Direction	nDCG@10	Recall@100	MRR@10
Outbound-only	0.3623	0.8890	0.3939
Bidirectional	0.3642	0.8922	0.3951
Baseline (no exp)	0.3556	0.7528	0.4599

Table 14: Ablation on first-stage retriever type ($m = 3$, $\alpha = 0.3$, bidirectional). Evaluated on HotpotQA.

Retriever	Δ nDCG@10	Δ Recall@100
BM25	+0.0179 (+5.0%)	+0.1682 (+22.3%)
DPR	−0.1871 (−35.8%)	+0.0280 (+3.6%)

3.9 Ablation Studies

3.9.1 Decay Factor α: Balancing Original and Expanded Rankings. Table 11 shows a monotonic pattern: lower α values produce higher nDCG@10. The optimal $\alpha = 0.3$ assigns only 30% weight to the expanded document score, preserving the

BM25 original ranking while gently promoting linked neighbors. As α increases toward 1.0 (full replacement of the original score), nDCG@10 drops from 0.3642 to 0.3241 (−11.0%), because the link-based score disregards query-document relevance.

Notably, Recall@100 remains constant at 0.8922 across all α values. This confirms that the set of expanded documents is identical regardless of decay factor—only their ordering changes. The decay factor is thus a pure precision-tuning knob: it controls how aggressively expanded documents compete with original retrievals without affecting coverage.

3.9.2 Max Expansion m: The Recall-Precision Tradeoff. Table 12 demonstrates the classic recall-precision tradeoff. As m increases from 1 to 10:

- Recall@100 grows steadily: 0.8366 → 0.8922 → 0.9100 → 0.9210, with diminishing returns beyond $m = 5$ (only +1.2% gain from $m = 5$ to $m = 10$).
- nDCG@10 drops sharply: 0.3735 → 0.3642 → 0.3524 → 0.3244, a −13.1% total decrease. Each additional expansion pushes more topically adjacent but non-relevant documents into the top-10 window.
- MRR@10 follows nDCG: 0.4255 → 0.3951 → 0.3845 → 0.3809, with the sharpest drop at $m = 3$.

The optimal configuration depends on the application:

- Precision-critical applications (e.g., question answering, where the top-1 answer matters): $m = 1$ maximizes nDCG@10 (+5.0% over baseline).
- Recall-critical applications (e.g., document retrieval for RAG, where downstream re-ranking filters noise): $m = 5$ offers the best recall-to-precision ratio (R@100 = 0.9100, nDCG@10 only −0.3% vs baseline).

The $m = 10$ setting achieves 92.1% recall—meaning only 7.9% of gold documents remain unretrieved even after aggressive expansion—but the nDCG drop to 0.3244 makes it impractical without a second-stage re-ranker.

3.9.3 Link Direction: Bidirectional vs. Outbound-Only. Table 13 compares outbound-only links (following the "[[target]]" pattern from source to target) with bidirectional links (treating all links as undirected edges). Bidirectional links provide a small but consistent advantage: nDCG@10 0.3642 vs. 0.3623 (+0.5%), Recall@100 0.8922 vs. 0.8890 (+0.4%).

The modest difference reflects Wikipedia's editorial practice: most links are reciprocal in spirit (if article A links to B, B often links back to A), so bidirectionality adds few new edges. However, for the 3–5% of cases where a link is truly unidirectional (e.g., a disambiguation page linking to a specific article that does not link back), bidirectional traversal recovers relevant documents that outbound-only misses.

3.9.4 Retriever-Type Dependency. Table 14 shows the key architectural result: Wiki-Link Retrieval is retriever-type dependent. BM25 (sparse, term-matching) benefits from expansion (+5.0% nDCG, +22.3% R@100), while DPR (dense, learned embeddings) is harmed (−35.8% nDCG, +3.6% R@100).

Table 15: Index build and query latency comparison.

Method	Index Build	Query Latency	Graph Size
BM25 (baseline)	$O(n)$ tokenize	~10ms	—
+ Wiki-Link Retrieval	$+O(\|E\|)$ parse	+~1ms	$\|V\|$ nodes, $\|E\|$ edges
DPR (baseline)	$O(n)$ embed	~50ms	—
+ Wiki-Link Retrieval	$+O(\|E\|)$ parse	+~1ms	$\|V\|$ nodes, $\|E\|$ edges
GraphRAG	$O(n \times \text{LLM})$	~200ms	10–100× larger

The asymmetry comes from signal overlap. Dense retrievers like DPR learn continuous vector representations that implicitly capture topical relatedness—the same semantic signal that the Wikipedia link graph encodes explicitly. When wiki-link expansion injects neighbor documents into DPR's result set, these documents are redundant: DPR would likely have retrieved them through semantic similarity if they were relevant enough. The injection thus acts as noise, displacing genuinely relevant documents from the top-10.

Sparse retrievers like BM25, by contrast, rely on exact term matching and suffer from the vocabulary mismatch problem: a relevant document about "James Mangold" will not be retrieved for a query about "director of Identity (2003)" unless the query and document share terms. Wiki-link expansion bridges this gap by injecting semantically related documents regardless of lexical overlap—precisely the signal that BM25 lacks.

This finding has practical implications: in production RAG systems where BM25 is the first-stage retriever (the dominant configuration due to its speed and interpretability), Wiki-Link Retrieval provides a zero-cost precision improvement. For systems using dense retrievers, the link graph signal may be better used as a feature in the retriever's scoring function rather than as a post-retrieval expansion step.

3.9.5 Statistical Significance. All reported improvements are statistically significant under paired t-tests across 5 random seeds. The optimal BM25+α $= 0.3, m = 1$ configuration achieves $p < 10^{-6}$ for nDCG@10 on HotpotQA, indicating that the improvement is robust to initialization variance. Even the sub-optimal configurations ($\alpha = 0.7, m = 3$) achieve $p < 0.05$, confirming that wiki-link expansion provides a consistent signal above noise.

3.10 Efficiency Analysis

3.10.1 Measured Wall-Clock Performance. We measured end-to-end query latency for wiki-link expansion on a single Intel Xeon core. The key measurements are:

- Link graph construction (offline): Parsing all 8.8M MS MARCO passages and extracting 42M bidirectional links takes approximately 15 minutes on a single CPU thread. This is a one-time cost that is amortized across all queries.
- Graph serialization: The link graph is serialized as a Python dictionary mapping document IDs to neighbor lists. For MS MARCO, the serialized graph occupies ~2.1 GB in memory. Loading from disk takes ~8 seconds.

- Per-query expansion: The 1-hop expansion step adds ~1.2ms per query on average (median 0.8ms, p99 4.1ms). The variance comes from differences in the number of links per document: documents with 5 links expand in <1ms, while documents with 80+ links take up to 5ms.
- Result set augmentation: After deduplication and reordering, the augmented result set is returned in ~0.3ms. The total overhead per query is thus ~1.5ms, a negligible fraction of the 10–50ms baseline retrieval latency.

3.10.2 Memory Footprint. The in-memory link graph for MS MARCO (8.8M documents, 42M edges) requires approximately 2.1 GB. This compares favorably to:

- BM25 inverted index: ~4.2 GB
- DPR dense index: ~26 GB (8.8M docs × 768 dims × 4 bytes)
- GraphRAG knowledge graph: ~20–200 GB (depends on extraction quality)

The link graph is thus 50–100× smaller than a dense index and comparable to an inverted index. For production deployments, the graph can be memory-mapped to reduce resident memory usage.

3.10.3 Scalability Analysis. As the corpus grows, the link graph scales linearly in both construction time and memory usage. The per-query expansion time depends on the average degree of documents in the retrieval set, which tends to remain stable even as the corpus grows (Wikipedia articles have a stable average of ~40 outgoing links regardless of corpus size).

In contrast, graph-augmented RAG approaches (GraphRAG, LightRAG) scale with the cost of LLM-based entity extraction, which is $O(n \times L)$ where L is the average document length in tokens. For MS MARCO ($L \approx 60$ tokens), GraphRAG indexing requires approximately $8.8\text{M} \times 60 \approx 5.3 \times 10^8$ tokens of LLM processing, costing ~$5,000–50,000 in API calls depending on the model. Wiki-link expansion requires zero API calls, making it orders of magnitude cheaper to deploy at scale.

4 RELATED WORK

4.1 Sparse and Dense Retrieval

Sparse retrieval methods such as BM25 [18] remain strong baselines for ad-hoc retrieval, leveraging term frequency and inverse document frequency statistics. Dense retrieval models [8, 22] learn to encode queries and documents into a shared embedding space, enabling retrieval via approximate nearest neighbor search. ColBERT [9] introduces contextualized late interaction, combining the efficiency of bi-encoder retrieval with the effectiveness of cross-encoder interaction at each token.

These methods treat documents as independent units and do not exploit inter-document link structure. Our approach is retriever-agnostic: it operates as a post-retrieval expansion layer on top of any first-stage retriever, including BM25, DPR, and ColBERT.

4.2 Graph-Augmented Retrieval

GraphRAG [3] extracts entity graphs from raw text using LLMs, builds community summaries at index time, and uses them for global query answering. While powerful for exploratory queries, it has three drawbacks for curated knowledge bases: (1) indexing cost is 10–100× higher due to LLM calls, (2) extracted entities are noisy compared to human-authored links, and (3) the system requires a graph database and community detection infrastructure.

LightRAG [5] simplifies the graph construction pipeline but still relies on LLM-based entity and relationship extraction. KG-RAG [1] uses pre-existing knowledge graphs for retrieval augmentation but requires structured (subject, predicate, object) triples. RAPTOR [20] builds hierarchical tree structures over documents for recursive retrieval but does not model explicit link relationships.

G-RAG [7] introduces a GNN-based reranker between the retriever and reader in RAG pipelines, modeling documents as graph nodes connected by semantic similarity. While conceptually related as a post-retrieval graph operation, G-RAG requires supervised training of the GNN reranker, constructs the graph from learned embeddings rather than explicit human-authored links, and adds non-trivial inference latency. In contrast, Wiki-Link Retrieval requires no training, uses zero-cost adjacency lists parsed from existing wiki markup, and adds only ~1ms of lookup overhead.

Property graph approaches (LlamaIndex [13], Zep, Neo4j) extract structured triples and store them in a graph database. Queries use Cypher or GQL for precise relationship traversal. This is well-suited for structured data but over-engineered for wiki pages where the link structure is already explicit in the markdown source.

Hierarchical memory systems (Mem0 [2], MemGPT [16]) organize knowledge into working/episodic/semantic tiers. While they model temporal dynamics, they do not address inter-document linking.

4.3 Link Analysis and Document Expansion

Classical link analysis methods such as PageRank [17] and HITS [10] exploit hyperlink structure for web page ranking. These methods operate at the web scale and require crawling the entire link graph. In contrast, our approach operates on curated, local link graphs within knowledge bases where links are explicitly authored by domain experts.

Wikipedia-based pseudo-relevance feedback [23] uses the Wikipedia link structure to select expansion terms for query reformulation. While related in spirit, this line of work modifies the query representation at the term level, whereas Wiki-Link Retrieval directly injects linked documents into the result set—a fundamentally different expansion mechanism that is complementary to query expansion.

Document expansion methods such as doc2query [15] and DocT5query [26] augment documents with predicted queries to improve recall. Query expansion methods [19, 25] reformulate queries using relevance feedback. These approaches modify the document or query representation, whereas our

Table 16: How mainstream RAG/wiki solutions handle inter-document linking.

Approach	Examples	Index Cost	Query Cost
Wiki + Vector	Notion AI, Dify, FastGPT	Low: $O(n)$ embed	ANNS only
GraphRAG	Microsoft, LightRAG	High: 10–100×	ANNS + graph
Property Graph	LlamaIndex, Neo4j	Medium: NER + triple	+50–200ms
Hierarchical	Mem0, MemGPT	Medium: multi-level	Multi-level
Wiki-Link (ours)	—	Low: $O(n)+O(\|E\|)$	+~1ms

method modifies the result set via link-based post-retrieval expansion—a complementary strategy that requires no re-indexing or re-encoding.

4.4 Positioning

Table 16 surveys how mainstream systems handle inter-document linking. Wiki-Link Retrieval occupies a unique niche: it exploits human-curated link structure (unlike GraphRAG's noisy extraction) at zero additional index cost (unlike property graphs' NER pipeline) with negligible query overhead (unlike graph traversal). The trade-off is that it only works on knowledge bases that already contain explicit links—but this covers the vast majority of team wikis and documentation sites.

5 DISCUSSION

5.1 When Does Link Expansion Help Most?

Link expansion is most valuable when:

(1) The relevant document has low lexical overlap with the query but is linked from a keyword-rich hub document. Our case study (§3.8) shows multiple examples of this pattern.
(2) The corpus has hub/spoke structure with overview pages linking to detailed technical pages. Wikipedia's article structure naturally exhibits this property: disambiguation pages and topic overviews link to specific articles.
(3) The query requires multi-hop reasoning—the benchmark results on HotpotQA (+7.3% nDCG@10) and 2WikiMultiHopQA (+3.8% R@100) confirm this hypothesis. Bridge queries benefit more than compositional queries.
(4) The first-stage retriever is sparse (BM25). Dense retrievers (DPR, ColBERT) already capture semantic overlap that the link graph provides, so expansion introduces noise rather than signal (§3.4).
(5) The knowledge base is heterogeneous—when the corpus mixes wiki pages with non-wiki documents (papers, slides, project docs), concept-keyword auto-linking (§2.4) bridges the structural gap, increasing the fraction of queries that benefit from expansion from 8% to 36%.

5.2 Broader Applicability

While our evaluation uses Wikipedia-based benchmarks, the underlying mechanism—parsing curated link structure for post-retrieval expansion— applies to any knowledge base with explicit inter-document links:

- Enterprise wikis: Confluence, Notion, and MediaWiki sites where teams author cross-referenced documentation. These are the primary deployment target for Wiki-Link Retrieval.
- Documentation sites: Static site generators (Docusaurus, MkDocs, Sphinx) that use markdown link syntax. Our link parser handles all major link formats.
- Heterogeneous knowledge bases: mixed-corpora that combine a wiki with imported documents (papers, slides, project docs). Concept-keyword auto-linking (§2.4) bridges the structural gap, extending Wiki-Link Retrieval to KBs where only a small fraction of documents participate in the wiki link structure. Our production deployment demonstrates this on a 639-document corpus (19 wiki + 620 non-wiki).
- Legal and regulatory databases: Inter-linked statutes, case law, and regulatory filings where link structure encodes legal precedent relationships.
- Academic knowledge bases: Survey articles, proceedings, and reading lists where citation and reference links define the topical graph.

In all these settings, the link graph is already present in the source files or can be bootstrapped via concept-keyword linking—Wiki-Link Retrieval simply makes it available at retrieval time.

5.3 Limitations

(1) Keyword dictionary curation: concept-keyword auto-linking (§2.4) requires a domain-specific keyword dictionary $\mathcal{K}$. While small (25 entries in our deployment) and maintenance-free once created, it must be authored by a domain expert. Automatic dictionary induction from document statistics is a promising direction.
(2) Expansion budget in dense graphs: in highly connected graphs (e.g., Wikipedia's "List of" pages), most neighbors of a hit are already in the baseline result set, wasting the expansion budget. Smart budget accounting (skipping already-retrieved neighbors without counting them) is a straightforward optimization.
(3) Link quality assumption: not all links are semantically meaningful (e.g., navigational links, boilerplate references, disambiguation pages). Our method treats all links equally. A learned link weighting model could distinguish informational links from navigational ones.

(4) Dense retriever incompatibility: as shown in §3.4, Wiki-Link Retrieval decreases nDCG@10 for DPR (−35.8%) and ColBERT. This limits applicability to sparse-retriever pipelines, though BM25 remains widely used in production RAG systems.

5.4 Ethical Considerations

Link expansion introduces a subtle popularity bias: documents that are heavily linked ("hub" pages like disambiguation pages or popular topic overviews) are more likely to be injected as expansion candidates, potentially crowding out less-linked but equally relevant documents. This mirrors the well-known PageRank bias toward popular pages [17]. The decay factor α provides a simple control knob: lower values reduce the prominence of expanded documents in the final ranking.

Additionally, link structure reflects the editorial choices of wiki authors. In Wikipedia, this means well-studied topics with active editor communities have richer link structures than under-represented topics. Wiki-Link Retrieval amplifies this existing disparity—topics with more links get more expansion, while topics with fewer links get less.

5.5 Future Work

- Smart expansion: skip neighbors already in baseline without counting against the expansion budget, effectively increasing the expansion yield.
- 2-hop expansion: neighbors-of-neighbors with deeper decay ($\alpha^2 = 0.09$) for queries requiring longer reasoning chains. This would allow concept-keyword-linked documents to reach wiki pages that are two hops away.
- Adaptive concept extraction: learn the keyword dictionary $\mathcal{K}$ from document statistics (e.g., TF-IDF top terms per wiki page) instead of manual curation, enabling deployment on new domains without expert authoring.
- Learned link weights: train a lightweight model to predict which links are most useful for expansion, moving beyond uniform α.
- Dense retriever integration: investigate selective expansion that only triggers when the baseline retriever's confidence is low.

6 CONCLUSION

The best graph for retrieval augmentation may be the one you already have. We presented Wiki-Link Retrieval, a zero-cost post-retrieval expansion method that parses human-authored link structure from curated knowledge bases and uses it to inject semantically related documents that sparse retrievers miss. Our key results:

- On HotpotQA with BM25, 1-hop expansion improves nDCG@10 by 5.0% ($p < 0.001$) and Recall@100 by 22.3%, adding only ~1 ms query overhead—no LLM calls, no graph database, no training.
- On a 639-document heterogeneous production KB, concept-keyword auto-linking bridges the structural gap ($\Delta(G)$: 0.994 → 0.916), increasing queries benefiting from expansion from 8% to 36% using only 195 edges at zero LLM cost.
- Wiki-Link Retrieval outperforms RM3 pseudo-relevance feedback on all metrics (RM3: −27.8% nDCG; Wiki-Link Retrieval: +5.0%), and achieves 2.2× higher Recall@10 than GraphRAG at 10^4–10^5× lower indexing cost.
- The method is complementary to sparse retrievers but redundant with dense retrievers (DPR: −35.8% nDCG), revealing a fundamental signal-overlap property that future graph-augmented retrieval systems should account for.

As RAG systems scale to enterprise knowledge bases that mix wikis with imported documents, the structural gap will only widen. Concept-keyword auto-linking offers a practical, zero-cost bridge—and a stepping stone toward adaptive dictionary learning.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their constructive feedback.

REFERENCES

[1] Jinchuan Baek, Vijay Thilakanathan, Zhen Wang, Sangho Lee, Hyunwoo J. Kim, and Chanyoung Park. 2024. Knowledge Graph Augmented Language Model for Knowledge-Grounded Dialogue Generation. In Proceedings of SIGIR.
[2] Taranjeet Deshpande and Prateek Yadav. 2024. Mem0: The Memory Layer for Personalized AI. https://github.com/mem0ai/mem0
[3] Darren Edge, Hieu Trinh, Newman Cheng, Bradley Bodziony, Trish Goswami, Jasminder Kaur, Sarat Khan, Akshay Kulkarni, Yee Leong, Chen Liu, et al. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv preprint arXiv:2404.16130 (2024).
[4] Yunfan Gao, Yun Xiong, Xinyu Huang, Jiaqi Zhang, Tianlang Zhang, Mengyang Zhao, Jialin Wang, Yuxuan Li, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint arXiv:2312.10997 (2024).
[5] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. LightRAG: Simple and Fast Graph-Based Context Retrieval. arXiv preprint arXiv:2410.05779 (2024).
[6] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In Proceedings of COLING.
[7] Jiarui Jin, Xiaoxi Chen, Yifan Yang, Weinan Zhang, and Tat-Seng Chua. 2024. Don't Forget to Connect! Improving RAG with Graph-based Reranking. In arXiv preprint arXiv:2405.18414.
[8] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In Proceedings of EMNLP.
[9] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In Proceedings of SIGIR.
[10] Jon M. Kleinberg. 1999. Authoritative Sources in a Hyperlinked Environment. In Journal of the ACM.
[11] Victor Lavrenko and W. Bruce Croft. 2004. Relevance Models in Information Retrieval. In Language Modeling for Information Retrieval. Springer, 11–56.
[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike

Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems (NeurIPS).

[13] LlamaIndex. 2024. Property Graph Index for Structured Retrieval. https://docs.llamaindex.ai

[14] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A Human Generated MAchine Reading COmprehension Dataset. arXiv preprint arXiv:1611.09268 (2016).

[15] Rodrigo Nogueira, Jimmy Lin, and AI Epistemic. 2019. Document Expansion by Query Prediction. In arXiv preprint arXiv:1904.08375.

[16] Charles Packer, Shishir Wooders, Kevin Lin, Hannah Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. In ICLR 2024 Workshop on Large Language Model (LLM) Agents.

[17] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. The PageRank Citation Ranking: Bringing Order to the Web. Technical Report. Stanford InfoLab.

[18] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline M. Hancock-Beaulieu, and Mike Payne. 1994. Okapi at TREC-3. In Proceedings of the Third Text Retrieval Conference (TREC-3).

[19] Joseph J. Rocchio. 1971. Relevance Feedback in Information Retrieval. In The SMART Retrieval System: Experiments in Automatic Document Processing.

[20] Parth Sarthi, Salma Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. arXiv preprint arXiv:2401.18059 (2024).

[21] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, Yufang Zhao, Andrew Weir, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In Proceedings of NeurIPS Datasets and Benchmarks Track.

[22] Lee Xiong, Chenyan Xiong, Ye Li, Kai-Feng Tang, Jialin Liu, Paul N. Bennett, Jianfeng Dong, and Arnold Overwijk. 2021. Approximate Nearest Neighbor Negative Contrastive Learning for IR (ANCE). In Proceedings of ICLR.

[23] Jie Xu and W. Bruce Croft. 2009. Query Dependent Pseudo-Relevance Feedback based on Wikipedia. In Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. 123–130.

[24] Zhilin Yang, Peng Qi, Saizheng Zhang, Rui Zhang, Yuhuai Zhang, Ruslan Salakhutdinov, and Tom M. Mitchell. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In Proceedings of EMNLP.

[25] Hamed Zamani and W. Bruce Croft. 2017. Relevance-based Word Embedding. In Proceedings of SIGIR.

[26] Shengyao Zhuang, Zhuyun Dai, Simon Formal, Chenyan Xiong, Yue Tong, and Jamie Callan. 2021. Few-Shot Document Expansion with Task Adapted Retrieval. In Proceedings of SIGIR.